Theory of Algorithms – Decrease and Conquer

Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za

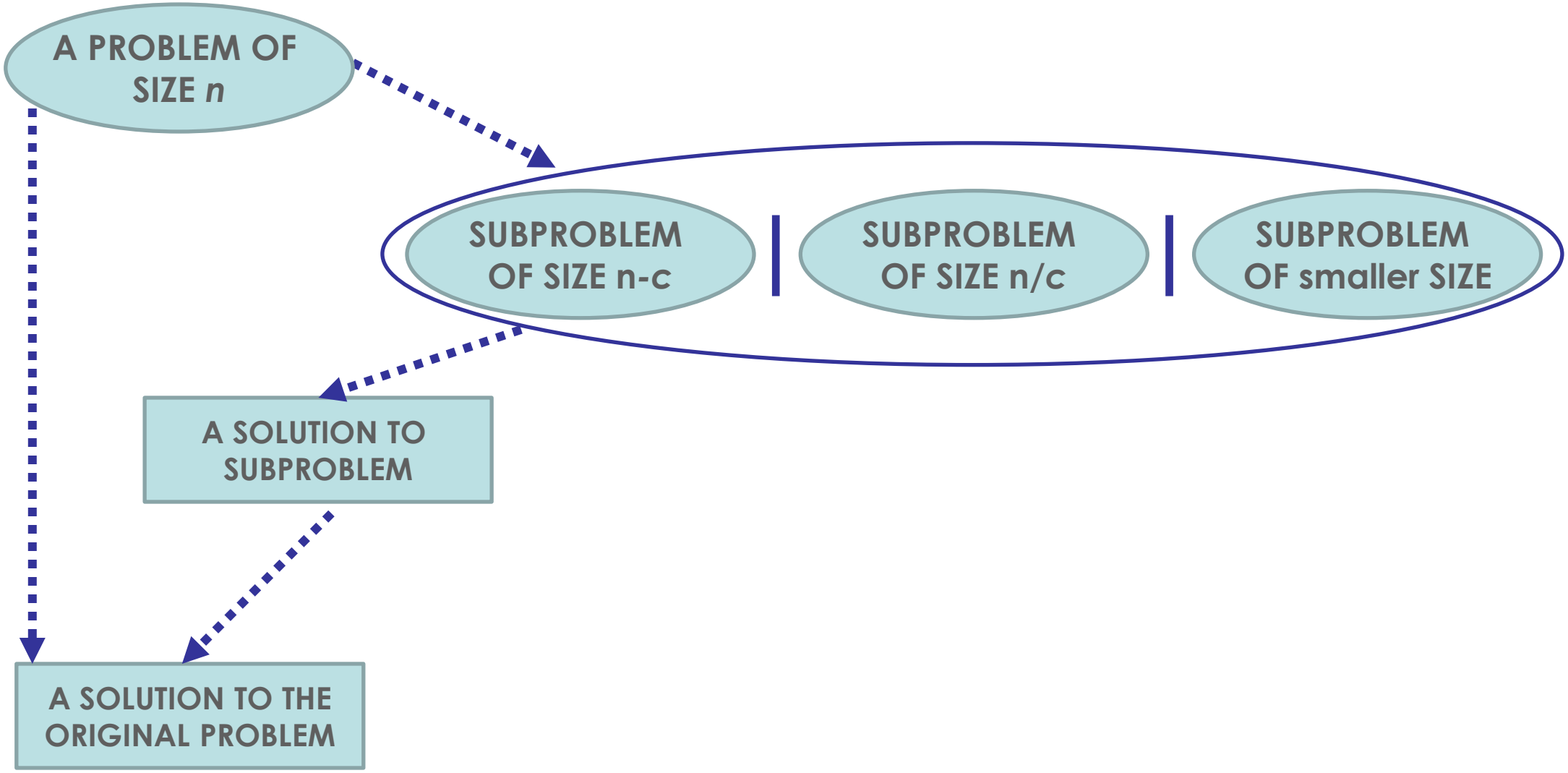




Introduction

- Strategy:
  - Solve smaller instance
  - Extend solution of smaller instance to obtain solution to original problem
- Also called inductive or incremental
- 3 Variants:
  - Decrease by a constant
  - Decrease by a constant factor
  - Variable size decrease

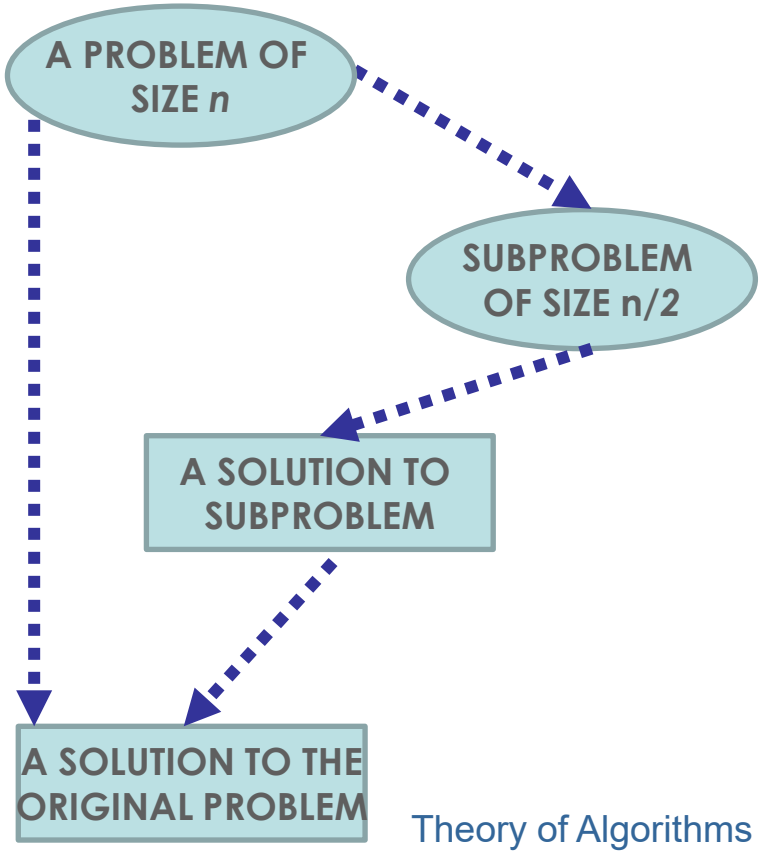
Variants Illustrated



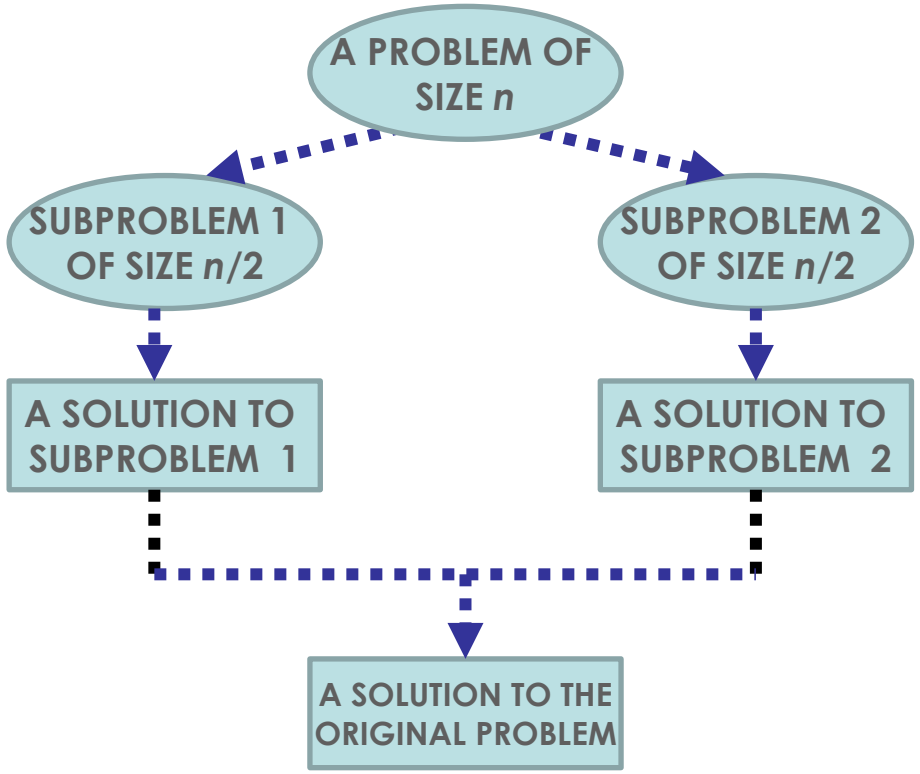
Divide and Conquer is Different

- Decrease and Conquer throws away half (or constant factor) of the work
- Divide and Conquer does both halves

Decrease by a Constant Factor and Conquer



Divide and Conquer



Map of Decrease and Conquer Algorithms

- Decrease by a constant (often 1):
  - Insertion sort
  - Graph Searching: DFS, BFS (**)
  - Generating permutations and subsets
  - Topological sort
- Decrease by a constant factor (usually 2):
  - Binary search (covered in CS1/2)
  - Fake-coin problem
  - Multiplication a la Russe
- Variable-size decrease:
  - Euclid's algorithm
  - Interpolation Search
  - Finding the k'th order statistic e.g. the median

Strengths and Weaknesses

Strengths:

- Can be implemented either top down (recursively) or bottom up (without recursion)
- Often very efficient (possibly $\Theta(\log n)$)

Weaknesses:

- Less widely applicable (especially decrease by a constant factor)

Theory of Algorithms – Decrease by a Constant





Insertion Sort

- Idea:
  - Assume an already sorted list of size *n-1 (for n input elements)*
  - Insert the remaining element in the correct position
  - Before doing this, recursively perform insertion sort on smaller list
- Recursive idea but better performed iteratively bottom up
- Decrease by 1 and conquer
- Worst case efficiency – $\Theta(n^2)$, but works well on partially sorted inputs

Insertion Sort Example

6 | **5** 0 2 8 7 4

5 6 | **0** 2 8 7 4

0 5 6 | **2** 8 7 4

0 2 5 6 | **8** 7 4

0 2 5 6 8 | **7** 4

0 2 5 6 7 8 | **4**

0 2 4 5 6 7 8

```
insertionSort(Array arr, int n)
    if (n <= 1)  return;  // base case

    insertionSort( arr, n-1 );

    // insert last element
    last = arr[n-1];  j = n-2;

    while (j >= 0 and arr[j] > last)
        arr[j+1] = arr[j];
        j = j - 1;

    arr[j+1] = last;
```

Insertion Sort Example

6	**5**	0	2	8	7	9	**sort(1);insert(2nd)**
5	**6**	**0**	2	8	7	9	**sort(2);**insert(3rd)
0	5	6	**2**	8	7	9	**sort(3);insert(4th)**
0	**2**	**5**	**6**	**8**	7	9	**sort(4);**insert(5th)
0	2	5	6	8	**7**	9	**sort(5);insert(6th)**
0	**2**	**5**	**6**	**7**	**8**	**9**	**sort(6);**insert(7th)
0	2	5	6	7	8	9	sort(7)

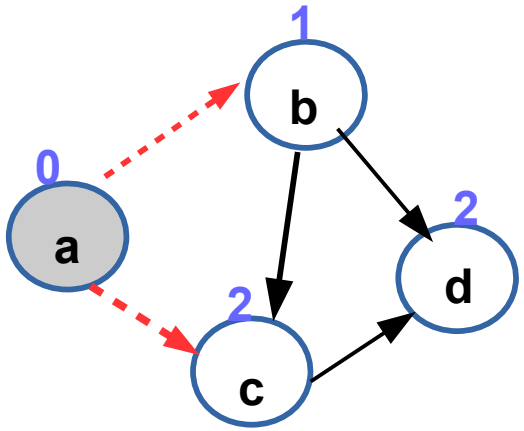
Topological Sort

- Problem:
  - In a directed acyclic graph, list the vertices in an order such that edge direction is respected
  - That is, for any directed edge the source vertex must appear before the destination in the list
  - Sometimes multiple possible orderings
  - Not solvable if there are cycles
  - Can use Depth-First Search (from CS2) to solve (see Levitin)
- Applications:
  - Ordering a set of courses that have pre-requisites
  - Evaluating formulae with dependencies in a spreadsheet
  - Ordering tasks in a complex project

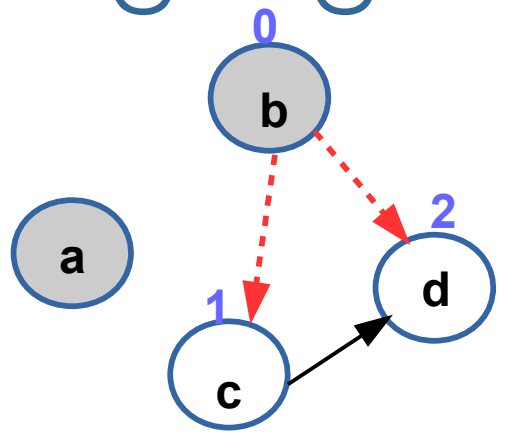
Topological Sorting Algorithm

- Decrease by 1 Solution:
  - Repeatedly remove a source vertex and its incident outgoing edges
  - Append to sorted list
  - If no source vertices exist, either finished or there is a cycle

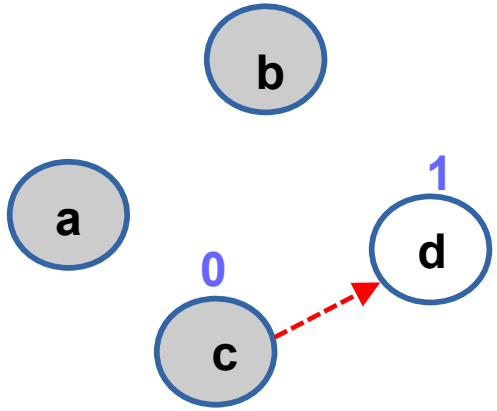
[numbers = number of incoming edges]



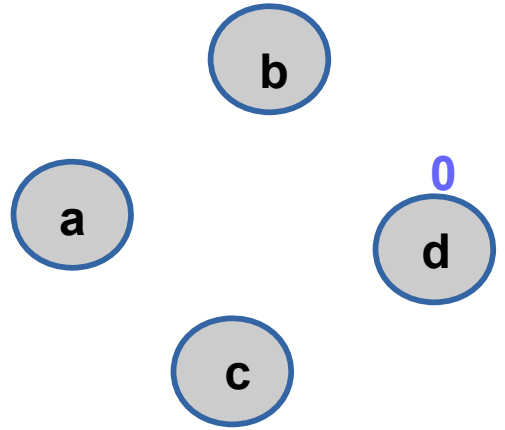
Remove **a** and edges to **b** and **c.** List {a}



Remove **b** and edges to **c** and **d.** List {a, b}



Remove **c** and edge to **d.** List {a, b, c}



Remove **d.** List {a, b, c, d}

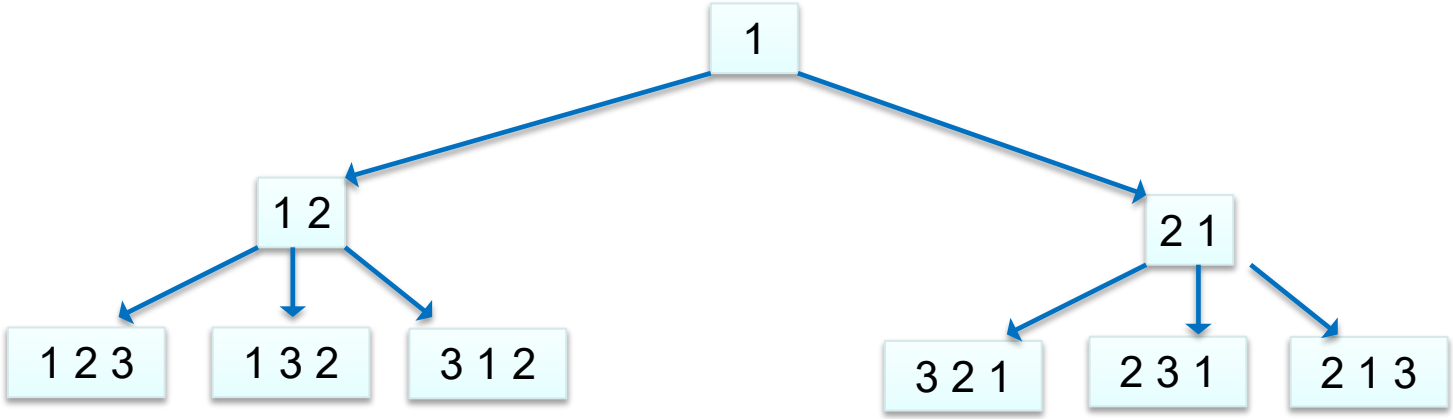
Generating the Powerset

- All possible 2^n subsets of set $\{a_1, a_2, \ldots, a_n\}$
  - Remember element order doesn't matter, and no duplicates
  - E.g. Powerset({a,b,c}) = [{ }, {a}, {b}, {a,b}, {c}, {a,c}, {b,c}, {a,b,c}]
- Solution via Decrease by 1 and conquer
  - Recursively generate subsets for $\{a_1, a_2, \ldots, a_{n-1}\}$ *with base case: { }*
  - Add a_n to each of these subsets
- Solution via direct enumeration of bit strings (brute force)
  - Enumerate all values from 0 … 2^n -1
  - Each set element is assigned to a bit position (from LSB to MSB is best)
  - If bit is on, then element is present, else absent
- We can find a minimal change powerset by using Binary Reflected Gray Code (see Levitin)
  - In terms of bit string rep, only single bit flips between each new subset

Generating Permutations

- Necessary as a component of Exhaustive Search
- All possible orderings of numbers in range {1, …, *n*}
  - Can be used as indices for other objects
- Solution:
  - Generate all *(n-1)!* permutations of *{1, …, n-1}*
  - Insert *n* into each possible position
  - During insert starting from the right or left *alternately*, satisfies Minimal-Change requirement (next permutation obtained by swapping two adjacent elements of previous)
  - Decrease by 1 and conquer

Generating Permutations Diagrammatically



- Example:
  - Start: *1*
  - Insert 2: *12* *21* *// insert from R*
  - Insert 3: *123 132 312* *321 231 213* *// ins from R, then L*

Johnson-Trotter Method

- Alternative permutation generator that avoids permutations of smaller lists
- Use arrows to keep track of what permutation comes next
  - An element k is ***mobile*** if its arrow points to an adjacent element smaller than it
  - → ← → ←

    3 2 4 1
  - 3 & 4 mobile. 1 & 2 not

Johnson-Trotter Algorithm

```
Initialize the first permutation with:
    ←    ←          ←
    1    2    …     n
    while last permutation has mobile elem:
         - find its largest mobile element k
         - swap k with neighbour it points to
         - reverse direction of elements > k
         - add the new permutation to the list
    Return the list of permutations
```

Johnson-Trotter Example

←	←	←
1	2	**3**
←	←	←
1	**3**	2
←	←	←
3	1	**2**
→	←	←
3	2	1
←	→	←
2	**3**	1
←	←	→
2	1	**3**

```
while last permutation
has mobile elem:
    - find its largest mobile element k
    - swap k with neighbour it points to
    - reverse direction of elements > k
    - add the new permutation to the list
```

Theory of Algorithms – Decrease by a Constant Factor





Fake Coin Problem

- Problem:
  - Among n coins, one is fake (and weighs less)
  - We have a balance scale which can compare any two sets
- Algorithm:
  - Divide into two size $\lfloor n/2 \rfloor$ piles (keeping a coin aside if n is odd)
  - If they weigh the same then the extra coin is fake
  - Otherwise proceed recursively with the lighter pile
- Efficiency:
  - $W(n) = W(\lfloor n/2 \rfloor) + 1$ for $n > 1$; $W(1) = 0$
  - $W(n) = \lfloor log_2\ n \rfloor = \Theta(\log_2 n)$
- Can we do better ?



Fake Coin Problem II

- Problem:
  - Among *n* coins, one is fake (and weighs less)
  - We have a balance scale which can compare any two sets
- Algorithm (3 pile version):
  - Divide into **three** piles ($\lceil n/3 \rceil$, $\lceil n/3 \rceil$, $n - 2\lceil n/3 \rceil$)
  - Weigh first two
  - If they weigh the same then discard, and continue with third pile
  - Otherwise proceed recursively with the lighter pile
- Efficiency:
  - $W(n) = W(\lceil n/3 \rceil) + 1$ for $n > 1$; $W(1) = 0$
  - $W(n) = \Theta(\log_3 n)$ [better by factor ~ 1.6 than two pile solution]

Multiplication a la Russe

- Solution:
  - n*m = (n/2) * (2m) if n is even
    ((n-1)/2) * (2m) + m if n is odd
  - Stop when n= 1
  - Using doubling and halving instead of multiplication is efficient in hardware (just shift operations!)
  - Decrease by a constant factor of 2 and conquer
- Example:
  - 50 * 20 = (25 * 40) = (12 * 80) + 40 = (6 * 160) + 40
    = (3 * 320) + 40 = (1 * 640) + 320 + 40 = 1000

Theory of Algorithms – Variable Size Decrease





Euclid's GCD

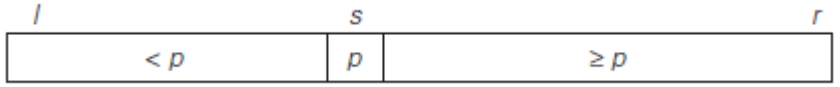
- Problem:
  - Greatest Common Divisor of two integers *m* and *n* is the largest integer that divides both exactly
- Euclid's Solution:
  - *gcd(m, n) = gcd(n, m mod n)*
  - *gcd(m, 0) = m*
  - Right-side args are smaller by neither a constant size nor factor
- Example:
  - gcd(60, 24) = gcd(24, 12) = gcd(12, 0) = 12

Finding k^{th} Order Statistic (k^{th} smallest element)

- Problem:
  - Find the kth smallest element in a list
  - Median is found when $k = \lceil n/2 \rceil$
  - Sorting the list may be inefficient

ALGORITHM *Quickselect*$(A[l..r], k)$
//Solves the selection problem by recursive partition-based algorithm
//Input: Subarray $A[l..r]$ of array $A[0..n-1]$ of orderable elements and
// integer k $(1 \le k \le r - l + 1)$
//Output: The value of the kth smallest element in $A[l..r]$
$s \leftarrow$ *LomutoPartition*$(A[l..r])$ //or another partition algorithm
if $s = k - 1$ **return** $A[s]$
else if $s > l + k - 1$ *Quickselect*$(A[l..s-1], k)$
else *Quickselect*$(A[s+1..r], k - 1 - s)$

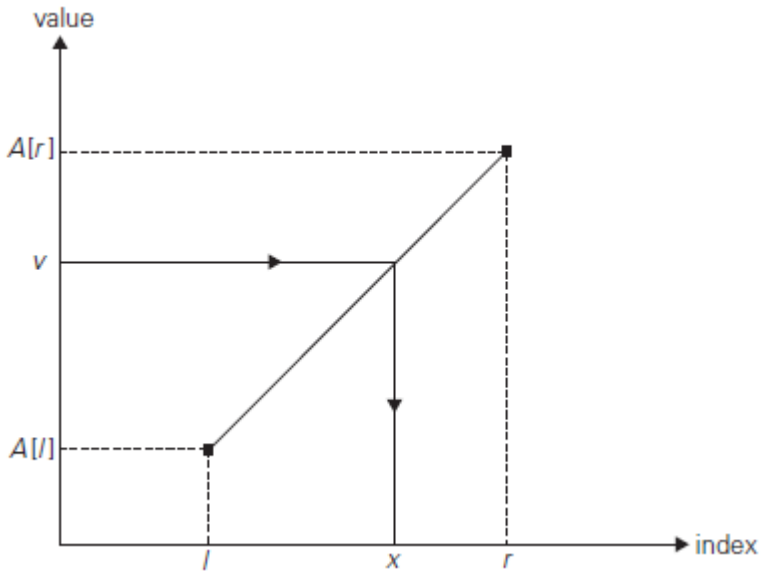
- Solution sketch:
  - Can use *partitioning scheme* to find a pivot p which partitions array



  - Partition as usual
  - If pivot == $(k-1)^{th}$ array position we are done, else we continue with one of the 2 partitions, depending on pivot
  - Basis of *quickselect* algorithm - average case $\Theta(n)$, worst case $\Theta(n^2)$

Interpolation Search

- Key search for sorted input
  - Alternate to even split (binary search)
- The way humans search through a phone book (look near the beginning for 'Bloggs')
- Assumes that array values between the leftmost (*A[l]*) and rightmost (*A[r]*) elements increase linearly
- Algorithm (key = *v*, find search index, x):
  - Setup straight line through (*l, A[l]*) and (*r, A[r]*)
  - Find point *P* on line at *y* = *v* → proposed index
  - If A[x] = key, stop else binary split on x
- On average better than binary search
- But worst case requires n key comparisons



$$x = l + \left\lfloor \frac{(v - A[l])(r - l)}{A[r] - A[l]} \right\rfloor$$